DAA IA – 1

Part A (5 × 2 = 10 Marks)

a. Can you list the steps involved in an algorithm with an example?

Steps in an algorithm:

1. Start: Begin the algorithm.
2. Input: Accept input data.
3. Process: Perform computations or decision-making steps.
4. Output: Display or return the result.
5. End: Terminate the algorithm.

Example – Find the largest of two numbers:

```
Step 1: Start
Step 2: Input a, b
Step 3: If a > b, then max = a
        Else max = b
Step 4: Print max
Step 5: End
```

b. Write the time complexity of the Quick sort algorithm in the best, average, and worst cases.

- Best Case: O(n log n) — When the pivot divides the array into two equal halves.
- Average Case: O(n log n) — On average over all pivots.
- Worst Case: $O(n^2)$ — When the pivot is always the smallest or largest element.

c. Define space complexity in the context of performance analysis.

Space complexity is the amount of memory space required by an algorithm to run as a function of the size of the input. It includes space for:

- Input data
- Auxiliary (temporary) space
- Recursion stack (if any)
- Output

d. Define linear search.

Linear Search is a simple searching algorithm that checks each element in a list sequentially until the desired element is found or the list ends.

Example:
For array `arr = [2, 4, 6, 8]`, to find `6`, it checks `2 → 4 → 6` and returns index 2.

e. Define Big O, Omega, and Theta notations.

- Big O (O): Upper bound — the worst-case time complexity.
- Omega (Ω): Lower bound — the best-case time complexity.
- Theta (Θ): Tight bound — the average/typical-case, or exact bound when best and worst cases are similar.

Part B (4 × 5 = 20 Marks)

1. Describe the key characteristics of an algorithm and explain why each is important.

An algorithm must have the following **key characteristics**:

1. **Input**:
    - It should accept **zero or more inputs**.
    - Importance: Enables processing of dynamic or real-world data.
2. **Output**:
    - It should produce at least **one output**.
    - Importance: Solves the given problem by providing a result.
3. **Finiteness**:
    - The algorithm must **terminate** after a finite number of steps.
    - Importance: Prevents infinite loops and ensures the solution is obtained.
4. **Definiteness**:
    - Every step must be **precisely and unambiguously defined**.
    - Importance: Ensures clarity and consistent execution.
5. **Effectiveness**:
    - Each step must be **basic enough** to be carried out by a person or machine.
    - Importance: Guarantees practical implementation.

2. Design an algorithm and express its time complexity using Big-O notation, explaining how it reflects its efficiency.

Problem: Find the **sum of elements** in an array of size `n`.

Algorithm:

```
Step 1: Start
Step 2: Input array A of size n
Step 3: Initialize sum = 0
Step 4: For i = 0 to n-1
```

```
            sum = sum + A[i]
Step 5: Output sum
Step 6: End
```

Time Complexity Analysis (Big-O):

- **Loop runs `n` times**, performing a constant-time addition in each iteration.
- Therefore, **Time Complexity = O(n)**

Explanation:

- Big-O notation represents the **upper bound** of the algorithm's runtime.
- In this case, the algorithm's time increases **linearly** with the size of the input array.
- Hence, it is **efficient** for small to medium-sized datasets, but may be slower for very large arrays.

3. Explain the Insertion Sort algorithm, including how it sorts elements and its overall working mechanism with an example program in Python.

Working Mechanism of Insertion Sort:

Insertion Sort builds the sorted list by repeatedly taking one element from the unsorted part and inserting it into the correct position in the sorted part.

Steps:

1. Start with the second element.
2. Compare it with elements to its left.
3. Shift larger elements to the right.
4. Insert the element in the correct position.
5. Repeat for all elements.

Example:

Given array: `[5, 3, 4, 1]`

- **Pass 1**: Key = 3 → Insert before 5 → `[3, 5, 4, 1]`
- **Pass 2**: Key = 4 → Insert between 3 and 5 → `[3, 4, 5, 1]`
- **Pass 3**: Key = 1 → Insert at the start → `[1, 3, 4, 5]`

Python Program:

```
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1
```

```
        # Shift elements of arr[0..i-1], greater than key
        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j -= 1

        arr[j + 1] = key

# Example usage
arr = [5, 3, 4, 1]
insertion_sort(arr)
print("Sorted array:", arr)
```

Output:

```
Sorted array: [1, 3, 4, 5]
```

4. Apply Binary Search to find a given element in a sorted array and compare its performance with Linear Search.

Binary Search:

Definition: Binary Search is an efficient algorithm used to find an element in a **sorted array** by repeatedly dividing the search interval in half.

Algorithm Steps:

1. Start with low = 0 and high = n-1.
2. Find mid = (low + high) // 2.
3. If arr[mid] == target, return index.
4. If arr[mid] > target, search in the left half.
5. If arr[mid] < target, search in the right half.
6. Repeat until low > high.

Python Example:

```
def binary_search(arr, target):
    low = 0
    high = len(arr) - 1

    while low <= high:
        mid = (low + high) // 2

        if arr[mid] == target:
            return mid
```

```
        elif arr[mid] < target:
            low = mid + 1
        else:
            high = mid - 1

    return -1

# Example usage
arr = [1, 3, 5, 7, 9, 11]
target = 7
index = binary_search(arr, target)
print("Element found at index:", index)
```

Linear Search:

Checks each element **one-by-one** from start to end.

```
def linear_search(arr, target):
    for i in range(len(arr)):
        if arr[i] == target:
            return i
    return -1
```

Performance Comparison:

Feature	Binary Search	Linear Search
Time Complexity	O(log n)	O(n)
Requires Sorting	Yes	No
Efficiency	Very fast for large, sorted arrays	Slower for large arrays
Best Case	O(1)	O(1)
Worst Case	O(log n)	O(n)

---

5. Explain the concept of space and time complexity in the performance analysis of algorithms.

Time Complexity:

- **Definition**: Time complexity measures the **amount of time** an algorithm takes to run as a function of the input size n.
- It helps evaluate the **speed or performance** of an algorithm.
- Expressed using **asymptotic notations** like:
  - **O(n)** – Worst-case
  - **Ω(n)** – Best-case

- **Θ(n)** – Average-case or tight bound

Example:

- A loop running n times has **O(n)** time complexity.
- Nested loops often have **O(n^2)**.

Space Complexity:

- **Definition**: Space complexity is the **amount of memory** required by an algorithm as a function of the input size n.
- Includes:
  - Input storage
  - Temporary variables
  - Recursion stack (if any)
  - Output (if stored)

Example:

- A function using a single variable has **O(1)** space.
- A function using an auxiliary array of size n has **O(n)** space.

Importance in Performance Analysis:

- **Time complexity** helps choose faster algorithms.
- **Space complexity** helps ensure the algorithm fits within memory constraints.
- Together, they help developers **balance speed and memory usage** for optimal performance.

Part C (2 × 10 = 20 Marks)

1. Analyze the differences between Big O, Omega, and Theta notations and determine when each should be used to express algorithm performance.

1. Big O Notation (O) – Upper Bound

- Describes the **worst-case** time or space an algorithm may take.
- It gives the maximum number of steps the algorithm will perform.
- Used when we want to guarantee that the algorithm will **not exceed** a certain time.

Example:
A loop that runs up to n times → **O(n)**
If a sorting algorithm takes up to n^2 steps → **O(n^2)**

2. Omega Notation (Ω) – Lower Bound

- Describes the **best-case** time or space an algorithm may take.

- Indicates the minimum amount of resources the algorithm will need.
- Used when analyzing the **best possible** performance.

Example:
If a linear search finds the element at the start → **Ω(1)**
Because only one comparison is needed.

3. Theta Notation (Θ) – Tight Bound

- Describes the **average-case** or exact bound.
- Indicates that the algorithm will take **roughly the same amount of time** in all cases.
- It gives both upper and lower bounds: **Θ(f(n)) = O(f(n)) ∩ Ω(f(n))**

Example:
If a loop always runs exactly n times regardless of input → **Θ(n)**

Comparison Summary:

Notation	Describes	When Used
O(f(n))	Worst-case	To ensure performance doesn't exceed a limit
Ω(f(n))	Best-case	To analyze minimal resource usage
Θ(f(n))	Average/tight-case	When performance is stable and predictable

---

Conclusion:

- Use **Big O** to ensure reliability in worst-case.
- Use **Omega** for understanding optimal conditions.
- Use **Theta** when performance is **consistently predictable**.

2. Describe the Quick Sort algorithm including the concept of pivot and partitioning, and write the C program for Quick Sort.

Quick Sort Algorithm:

Quick Sort is a **divide-and-conquer** sorting algorithm that works by selecting a **pivot** element, partitioning the array around the pivot, and then recursively sorting the subarrays.

Key Concepts:

1. **Pivot**: An element chosen from the array to divide the data into two parts.
2. **Partitioning**: Reorganizing the array so that:
    - Elements **less than** pivot are on the **left**
    - Elements **greater than** pivot are on the **right**
3. Recursively apply the process to the left and right subarrays.

Steps of Quick Sort:

1. Choose a pivot element (commonly the last element).

2. Partition the array using the pivot.
3. Recursively apply Quick Sort to the subarrays before and after the pivot.

C Program for Quick Sort:

```
#include <stdio.h>

void swap(int* a, int* b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

int partition(int arr[], int low, int high) {
    int pivot = arr[high]; // pivot element
    int i = (low - 1);     // index of smaller element

    for (int j = low; j <= high - 1; j++) {
        // If current element is smaller than pivot
        if (arr[j] < pivot) {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }

    swap(&arr[i + 1], &arr[high]); // place pivot in correct position
    return (i + 1);
}

void quickSort(int arr[], int low, int high) {
    if (low < high) {
        // partitioning index
        int pi = partition(arr, low, high);

        quickSort(arr, low, pi - 1);  // Left of pivot
        quickSort(arr, pi + 1, high); // Right of pivot
    }
}
```

```
int main() {
    int arr[] = {10, 7, 8, 9, 1, 5};
    int n = sizeof(arr)/sizeof(arr[0]);

    quickSort(arr, 0, n - 1);

    printf("Sorted array: ");
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);

    return 0;
}
```

Time Complexities:

Case	Time Complexity
Best	O(n log n)
Average	O(n log n)
Worst	$O(n^2)$ (if pivot is poorly chosen)

---

3. Write a C program to implement the Merge Sort algorithm using recursion to sort an array of integers.

Merge Sort Algorithm Overview:

Merge Sort is a **divide-and-conquer** algorithm that:

1. **Divides** the array into two halves recursively until each subarray has one element.
2. **Merges** the subarrays back while sorting them.

Key Steps:

1. Divide the array into two halves.
2. Recursively sort both halves.
3. Merge the sorted halves into one sorted array.

C Program for Merge Sort:

```
#include <stdio.h>

void merge(int arr[], int left, int mid, int right) {
    int i, j, k;
    int n1 = mid - left + 1;
```

```
int n2 = right - mid;

// Create temporary arrays
int L[n1], R[n2];

// Copy data to temp arrays
for (i = 0; i < n1; i++)
    L[i] = arr[left + i];
for (j = 0; j < n2; j++)
    R[j] = arr[mid + 1 + j];

// Merge the temp arrays back
i = 0; // Initial index of L[]
j = 0; // Initial index of R[]
k = left; // Initial index of merged subarray

while (i < n1 && j < n2) {
    if (L[i] <= R[j]) {
        arr[k] = L[i];
        i++;
    }
    else {
        arr[k] = R[j];
        j++;
    }
    k++;
}

// Copy remaining elements of L[], if any
while (i < n1) {
    arr[k] = L[i];
    i++;
    k++;
}

// Copy remaining elements of R[], if any
```

```
    while (j < n2) {
        arr[k] = R[j];
        j++;
        k++;
    }
}

void mergeSort(int arr[], int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;

        // Sort first and second halves
        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);

        // Merge the sorted halves
        merge(arr, left, mid, right);
    }
}

int main() {
    int arr[] = {12, 11, 13, 5, 6, 7};
    int n = sizeof(arr) / sizeof(arr[0]);

    mergeSort(arr, 0, n - 1);

    printf("Sorted array: ");
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);

    return 0;
}
```

Time and Space Complexity:

Case	Time Complexity	Space Complexity
Best	O(n log n)	O(n)
Average	O(n log n)	O(n)

Case	Time Complexity	Space Complexity
Worst	O(n log n)	O(n)

IA - 2

Part A (5 × 2 = 10 Marks)

1. What is Greedy Method of Problem Solving? Mention 1 example of it.

- **Definition**: The **Greedy Method** solves problems by making a series of choices, each of which looks best at the moment (locally optimal), hoping that these choices lead to a global optimum.
- **Example**: **Activity Selection Problem** — selecting the maximum number of non-overlapping activities based on their start and end times.

2. What is the objective of the Traveling Salesperson Problem (TSP)?

- **Objective**: To find the **shortest possible route** that visits each city **exactly once** and returns to the starting city.
- It is a classic **NP-hard** problem in combinatorial optimization.

3. What is the basic idea behind Dynamic Programming? How is it different from recursion?

- **Basic Idea**: Break a problem into **overlapping subproblems**, solve each **once**, and store their solutions (using a table).
- **Difference from Recursion**:
  - Recursion recomputes subproblems.
  - DP **stores and reuses** subproblem results (**memoization** or **tabulation**).

4. What is a Minimum Cost Spanning Tree? Give one real-life application.

- **Definition**: A **spanning tree** of a connected graph that connects all vertices with the **minimum total edge weight**.
- **Real-life application**: Designing **cost-efficient electrical or computer networks** (e.g., laying out cables between servers).

5. What is Divide and Conquer Method of Problem Solving? Mention 1 example.

- **Definition**: Divide a problem into **smaller subproblems**, **solve each recursively**, and **combine** their results.
- **Example**: **Merge Sort** — divides the array, recursively sorts each half, and merges them.

Part B (4 × 5 = 20 Marks)

1. Explain Kruskal's algorithm for finding a minimum cost spanning tree. Illustrate one iteration with an example graph.

☑ **Kruskal's Algorithm Overview:**

- A **Greedy Algorithm** used to find the **Minimum Cost Spanning Tree (MST)** of a connected, undirected, weighted graph.
- It **sorts all edges by weight** and adds the smallest edge that doesn't form a cycle.

🔄 **Steps:**

1. Sort all edges in **non-decreasing order** of weight.
2. Initialize a forest where each vertex is a separate tree (using Disjoint Set/Union-Find).
3. Pick the smallest edge. If it connects **different trees**, add it to the MST.
4. Repeat until the MST contains **(V - 1)** edges.

▨ **Example Graph:**

Vertices: A, B, C, D
Edges with weights:

- AB – 1
- AC – 3
- BD – 4
- CD – 2
- AD – 5

Step-by-Step:

- Sort edges: AB(1), CD(2), AC(3), BD(4), AD(5)
- Initially, MST = {}
- Add **AB(1)** → no cycle
- Add **CD(2)** → no cycle
- Add **AC(3)** → connects A and C (already in AB) → **cycle** → skip
- Add **BD(4)** → connects B and D → no cycle → add

Final MST edges: **AB(1), CD(2), BD(4)**
Total cost = 7

2. Explain the principle of optimal substructure with respect to the All-Pairs Shortest Path problem.

☑ **What is Optimal Substructure?**

- A problem has **optimal substructure** if an **optimal solution** to the problem contains **optimal solutions to its subproblems**.

- It is a **key property** in dynamic programming.

🚗 **All-Pairs Shortest Path Problem:**

- Goal: Find the **shortest path** between every pair of vertices in a weighted graph.
- A classic algorithm for this is **Floyd–Warshall Algorithm**.

🔁 **How Optimal Substructure Applies:**

Let's say you want the **shortest path from vertex `i` to vertex `j`**.
Assume the path passes through an intermediate vertex `k`.
Then:

ShortestPath(i,j) = min(ShortestPath(i,j) , ShortestPath(i,k) + ShortestPath(k,j))

This equation shows that the best path from `i` to `j` **depends on** optimal paths from `i` to `k` and from `k` to `j`.

📌 **Example:**

Graph:

- A → B = 2
- B → C = 3
- A → C = 10

Now:

- Shortest(A, C) = min(10, A→B + B→C) = min(10, 2 + 3) = **5**

So, the optimal solution (A→B→C) is composed of two optimal subpaths (A→B and B→C).

🧠 **Conclusion:**

- **All-Pairs Shortest Path** problem demonstrates optimal substructure clearly.
- This property allows us to build solutions dynamically using the **Floyd-Warshall** or **repeated Dijkstra's** algorithms.

3. Write the backtracking approach for solving the N-Queen Problem. Explain how the solution space is generated.

☑️ N-Queen Problem Overview:

- Place **N queens** on an **N×N chessboard** so that **no two queens attack each other**.
- A queen can attack another in the **same row**, **same column**, or **diagonals**.

🔄 Backtracking Approach:

- Place queens **one row at a time**.
- For each row, try placing the queen in all **safe columns**.
- If placing the queen leads to a conflict, **backtrack** and try the next column.
- If a solution is found for all rows, it's a valid configuration.

🧱 Algorithm Steps:

1. Start from the first row.
2. For each column in the row:
    - Check if placing the queen is **safe**.
    - If **safe**, place the queen and recurse for the next row.
    - If it leads to a dead-end, **remove the queen** (backtrack).
3. Repeat until all queens are placed.

🧮 How Solution Space is Generated:

- Each node in the recursion tree represents a **board state**.
- Each level of the tree represents a row.
- Each branch corresponds to placing a queen in one of the available columns.

For `N=4`, you explore possibilities like:

```
Row 0: Try column 0 → next → Try column 1 → ...
Row 1: For each success in Row 0, try safe columns in Row 1, and so on.
```

🧷 Python-Style Pseudocode:

```
def is_safe(board, row, col, n):
    for i in range(row):
        if board[i] == col or \
           abs(board[i] - col) == abs(i - row):
            return False
    return True

def solve_n_queens(board, row, n):
    if row == n:
        print(board)  # solution found
        return
    for col in range(n):
        if is_safe(board, row, col, n):
            board[row] = col
            solve_n_queens(board, row + 1, n)
            board[row] = -1  # backtrack

n = 4
board = [-1] * n
```

```
solve_n_queens(board, 0, n)
```

🧠 Conclusion:

- **Backtracking** explores all possibilities efficiently.
- The **solution space** is pruned by rejecting unsafe positions early, avoiding brute-force.

4. Differentiate between FIFO Branch and Bound and LC (Least Cost) Branch and Bound methods with a small example or application area.

☑ Branch and Bound Overview:

- A method to solve **combinatorial optimization problems**.
- It builds a **state-space tree** and explores **partial solutions**.
- Two common strategies:
  - **FIFO (First-In-First-Out)**
  - **LC (Least-Cost)**

🔁 1. FIFO Branch and Bound:

- Uses a **queue (FIFO)** to explore nodes **level-by-level** (like BFS).
- Nodes are **processed in the order they are generated**, regardless of cost.
- **No prioritization** based on cost.

Example Use Case: Solving a puzzle like **15-puzzle** using uniform cost search.

🔽 2. LC (Least-Cost) Branch and Bound:

- Uses a **priority queue** based on **cost estimates** (lower bound).
- **Node with the lowest cost** is explored first.
- Efficient for problems where **cost estimation is available**.

Example Use Case: **Travelling Salesperson Problem (TSP)** using **reduced cost matrix** as lower bound.

📊 Comparison Table:

Feature	FIFO Branch and Bound	LC Branch and Bound
Node Selection	First generated	Least cost (lowest bound)
Data Structure Used	Queue	Priority Queue (Min-Heap)
Efficiency	May explore many nodes	More efficient (guided search)
Example Application	15-Puzzle	Traveling Salesperson Problem

🧠 Conclusion:

- **FIFO** is easier but less efficient.
- **LC** is optimal when **lower-bound cost** estimation is possible.

5. What is the Reliability Design problem in dynamic programming? What are the inputs and goal of the problem?

☑ Reliability Design Problem Overview:

- The **Reliability Design Problem** is a classic **resource allocation** problem.
- It involves designing a **reliable system** by allocating limited resources to **redundant components**.

🎯 Objective / Goal:

To **maximize the overall system reliability** by deciding:

- How many **redundant units** (spares) of each component to include,
- Without exceeding the total **resource (cost/weight) constraint**.

📥 Inputs:

1. **n** = number of components in the system.
2. For each component:
    - **r_ij** = reliability of the component if `j` units are used.
    - **c_ij** = cost or resource usage if `j` units are used.
3. **Total available budget/resource (C)**.

🧠 Dynamic Programming Approach:

- Let `R(i, c)` be the **maximum reliability** achievable using first `i` components with cost ≤ `c`.
- Use recurrence to build up solution:

  R(i,c)=max(R(i−1,c−cij)×rij)R(i, c) = \max \left(R(i-1, c - c_{ij}) \times r_{ij} \right)

  for each possible `j` units used for the i-th component such that cost doesn't exceed `c`.

🛠 Applications:

- Designing **fault-tolerant systems** in aerospace, medical devices, and communications.
- **Maximizing system reliability** under strict cost, space, or energy constraints.

🧠 Conclusion:

- It’s a **combinatorial optimization** problem solved using **dynamic programming**.
- The key challenge is to **balance cost and reliability** by making optimal component-level decisions.